

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Iustin Rusu FAF-253

Report

Laboratory work n.1

of Object-Oriented Programming

Checked by:
Dominic Flocea, *university assistant*
FCIM, UTM

Chișinău – 2026

Table of Contents

1 Program Structure **1**

2 Classes and Objects **1**

3 Static and Instance Members **2**

4 Methods and Responsibilities **3**

5 Testing and Correctness **4**

6 Reflection **5**

Laboratory Report - Logger and Parentheses Validator

Programming language: Kotlin

I chose Kotlin at the recommendation of a friend. After playing with it for a bit and starting to learn the basics it's a very powerful and fun language to code in, I will definitely consider using it in the future aswell.

1. Program Structure

What is the responsibility of `Logger`?

The `Logger` class is responsible for managing application log output. Specifically, it:

1. Filters log entries against a globally configured minimum severity threshold (`LogLevel`).
2. Formats log strings by substituting placeholder tokens (`%t`, `%m`, `%n`, `%l`, `%r`) using a single-pass substitution engine.
3. Emits the resulting formatted message to output.

What is the responsibility of `ParenthesesValidator`?

The `ParenthesesValidator` class is responsible for verifying whether a sequence of parentheses is syntactically balanced according to proper ordering and nesting rules. Its sole domain responsibility is processing the string and returning a boolean verdict (`true` if valid, `false` otherwise).

How do the two classes collaborate?

The two classes collaborate through object composition and message passing. `ParenthesesValidator` owns a private instance of `Logger` named `"Parentheses"`. During the execution of `isValid()`, the validator notifies the logger about state transitions (such as discovering an opening parenthesis or resolving a matching pair) by calling its methods (`logger.trace()`, `logger.info()`, `logger.warn()`, `logger.error()`). The validator does not know how formatting, filtering, or printing works, it simply delegates observational reporting to the logger.

2. Classes and Objects

What is a class in your implementation?

In my implementation, a class acts as a blueprint defining the state and behavior of an abstraction. For example, `class Logger(val name: String)` defines what each logger instance stores (basically its name) and what operations it can perform (`log`, `info`, `warn`, etc.), without containing the runtime values of any specific logger instance itself.

Give an example of an object created from a class.

In `ParenthesesValidator`, an object is instantiated using the expression:

```
private val logger = Logger("Parentheses")
```

Here, `Logger` is the class blueprint, while ‘logger‘ is the concrete object instantiated in memory on the heap with the specific name attribute `"Parentheses"`.

What state does each object store?

- Each ‘Logger‘ object stores its individual `name: String` attribute on the heap.
- Each `ParenthesesValidator` object stores a reference to its own `Logger` instance.
- Notably, the algorithm's transient counter `openCount` is deliberately stored as a local stack variable inside `isValid()`, rather than an object attribute, ensuring that consecutive validation calls on the same validator object remain independent and do not leak state between runs.

3. Static and Instance Members

Which members of `Logger` are static, and why?

The static members of `Logger` are placed inside its `companion object`:

- `var level: LogLevel`: The minimum level threshold.
- `var format: String`: The shared output format string.
- `private val timeFormatter: DateTimeFormatter`: The shared time pattern (`HH:mm:ss`).
- `private val randomMessages: Map<LogLevel, List<String»`: The message pools for the `%r` placeholder.

These are static because logging policy, output format, and message pools are cross-cutting concerns that belong to the class as a whole. Storing them on the companion object allocates them once in memory and ensures a consistent configuration across all loggers.

Which member belongs to an individual logger instance?

`val name: String` belongs to each individual `Logger` object. This allows different loggers (such as `"Parentheses"` or `"redLogger"`) to tag their messages with their own identity while still adhering to the same global format and filtering level.

What happens when the logging level is changed? Does it affect every logger instance? Explain why. Include snippets.

When `Logger.level` is changed, it immediately affects every existing and future `Logger` instance. This occurs because the instances do not store their own level, during `log()`, each instance accesses the single shared memory address located in the companion object.

```
// example demonstrating cross instance static effect
val redLogger = Logger("Red")
val blueLogger = Logger("Blue")

Logger.level = LogLevel.WARN

// both instances immediately suppress TRACE and INFO
redLogger.info("Color red") // does not print
blueLogger.info("Color blue") // does not print
redLogger.warn("Color green") // prints
```

4. Methods and Responsibilities

What is the responsibility of the `log` method?

The `log` method executes two discrete tasks:

1. **Filtering** - It evaluates `if (msgLevel >= level)`. In Kotlin, enums implement `Comparable` based on declaration order (`TRACE < INFO < WARN < ERROR`), so ordinal comparison decides whether the entry meets the threshold.
2. **Dispatching** - If allowed, it delegates string assembly to the private `formatMessage()` method and writes the final line to the console.

Why does `ParenthesesValidator` use a logger instead of implementing logging itself?

This follows the **Single Responsibility Principle** and **Encapsulation**. `ParenthesesValidator` should only contain logic for validating syntax. If the validator also managed time formatting, string interpolation, and console I/O, changing the log format would require modifying and retesting the validator code. Delegating to `Logger` decouples algorithmic logic from presentation logic.

Identify one example of a method call between objects or classes in your program.

Inside `class ParenthesesValidator`:

```
logger.info("Matched pair '()' found (remaining unmatched: $openCount)")
```

Here, the `ParenthesesValidator` object sends a message to its collaborator `logger` by calling its `info` method with a descriptive string.

5. Testing and Correctness

Give one valid and one invalid input for the parentheses validator.

1. **Valid input:** `"(())((()())())"`
2. As characters are read left-to-right, `openCount` increments on `(` and decrements on `)`. The counter stays ≥ 0 at all times and terminates at exactly `0`, confirming all pairs are matched.
3. **Invalid input:** `")((()))"`
4. On the first character `')'`, `openCount` is currently `0`. The guard condition `if (openCount == 0)` triggers an immediate early exit, logging a warning/error and returning `false`.

Explain how the program determines whether the input is valid.

The method processes the string in $O(n)$ **time** and $O(1)$ **auxiliary space**:

1. It maintains an integer counter `openCount` initialized to 0.
2. For each `(`, `openCount` is incremented.
3. For each `)`, if `openCount == 0`, a closing parenthesis appeared without an opening match, the method immediately fails. Otherwise, `openCount` is decremented.
4. After scanning the entire string, the string is valid if and only if `openCount == 0`. If `openCount > 0`, unclosed opening parentheses remain.

Describe one test that helped you find or prevent an error.

1. **The order counterexample (`")("`):** A naive implementation that simply compares `count('(') == count(')')` would return `true` for `")("` (since 1 == 1). Testing `")("` verified that early termination on unmatched closing parentheses is mandatory.
2. 2. **Placeholder injection in format strings:** When testing a message containing `"%n"` (e.g. `"50%news"`), naive chained string replacements (`.replace("%m", ...).replace("%n", ...)`) would corrupt the message by turning `%news` into the logger's name. Testing this verified that a single-pass regular expression (`Regex("%[tmlnr])"`) was necessary to ensure user input is never reevaluated.

6. Reflection

What was the most important OOP concept you applied in this laboratory?

I think the most important concept was **differentiating between instance state and static**. Designing `Logger` required recognizing that an identity (`name`) belongs to an individual object on the heap, while global operational policies (`level`, ‘`format`, message pools) belong to the class itself. By using Kotlin's `companion object` I could understand clearly how objects can share configuration without tight coupling.

If you had more time, what would you improve in your design, and why?

The current validator only handles round parentheses `()`, allowing an integer counter. To support `[]`, `{}`, and `<>`, I would replace the integer counter with a generic `ArrayDeque<Char>` stack to verify that brackets close in strict Last-In-First-Out (LIFO) order.